

PRACTICAL WEEK – 4

TASK 1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#define NUM_CHILDREN 3
```

```
int main()
```

```
{
```

```
    pid_t child[NUM_CHILDREN];
```

```
    int status;
```

```
    printf("Parent Process Started\n");
```

```
    printf("Parent PID: %d\n\n", getpid());
```

```
    // Create 3 child processes
```

```
    for (int i = 0; i < NUM_CHILDREN; i++)
```

```
    {
```

```
        child[i] = fork();
```

```
        if (child[i] < 0)
```

```
        {
```

```
            perror("fork failed");
```

```
            exit(EXIT_FAILURE);
```

```
        }
```

```

if (child[i] == 0)
{
    printf("Child %d started. PID = %d, Parent PID = %d\n",
        i + 1, getpid(), getppid());

    // Different completion times
    sleep((i + 1) * 2);

    printf("Child %d completed. PID = %d\n", i + 1, getpid());

    // Exit with different status
    exit(10 + i);
}
}

printf("\nAll 3 child processes have been created.\n");

// ----- WAIT -----

printf("\nUsing wait():\n");

pid_t finished = wait(&status);

if (finished == -1)
{
    perror("wait failed");
    exit(EXIT_FAILURE);
}

if (WIFEXITED(status))

```

```

{
    printf("wait() detected child PID %d finished with exit status %d\n",
        finished, WEXITSTATUS(status));
}

// ----- WAITPID -----

printf("\nUsing waitpid():\n");

for (int i = 0; i < NUM_CHILDREN; i++)
{
    // Skip the child already collected by wait()
    if (child[i] == finished)
        continue;

    pid_t pid = waitpid(child[i], &status, 0);

    if (pid == -1)
    {
        perror("waitpid failed");
        continue;
    }

    if (WIFEXITED(status))
    {
        printf("waitpid() detected child PID %d finished with exit status %d\n",
            pid, WEXITSTATUS(status));
    }
}

```

```

printf("\nParent process completed.\n");

return 0;
}

```

```

nikith@NIKITH:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

nikith@NIKITH:~$ nano week4_task1.c
nikith@NIKITH:~$ gcc week4_task1.c -o week4_task1
nikith@NIKITH:~$ ./week4_task1
Parent Process Started
Parent PID: 496

Child 1 started. PID = 497, Parent PID = 496
Child 2 started. PID = 498, Parent PID = 496

All 3 child processes have been created.

Using wait():
Child 3 started. PID = 499, Parent PID = 496
Child 1 completed. PID = 497
wait() detected child PID 497 finished with exit status 10

Using waitpid():
Child 2 completed. PID = 498
waitpid() detected child PID 498 finished with exit status 11
Child 3 completed. PID = 499
waitpid() detected child PID 499 finished with exit status 12

Parent process completed.
nikith@NIKITH:~$ |

```

TASK 2A

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

int main()
{
    pid_t pid;

```

```
pid = fork();

if (pid < 0)
{
    perror("fork failed");
    exit(EXIT_FAILURE);
}

if (pid == 0)
{
    // Child process
    printf("Child process: PID = %d\n", getpid());
    printf("Child PID = %d\n", getpid());

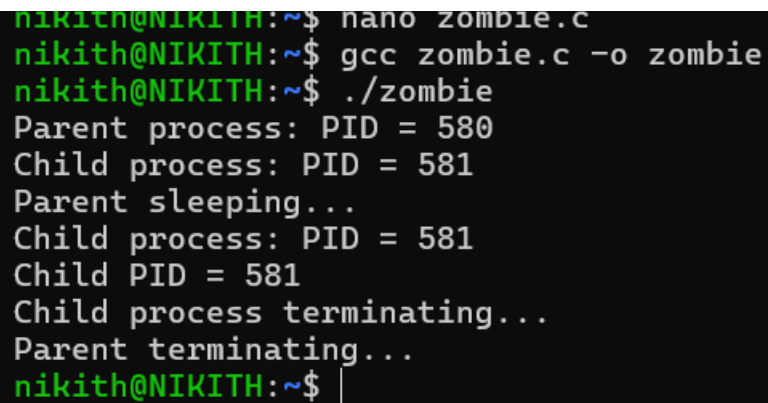
    printf("Child process terminating...\n");

    exit(0);
}
else
{
    // Parent process
    printf("Parent process: PID = %d\n", getpid());
    printf("Child process: PID = %d\n", pid);

    printf("Parent sleeping...\n");

    // Parent does NOT call wait()
    sleep(30);
}
```

```
    printf("Parent terminating...\n");  
}  
  
return 0;  
}
```



A terminal window with a black background and green text. The user 'nikith' is at the 'NIKITH' prompt. They run 'nano zombie.c', then 'gcc zombie.c -o zombie', and finally './zombie'. The program output shows the parent process (PID 580) spawning a child (PID 581), sleeping, and then terminating. The child process also terminates. The prompt returns to the user.

```
nikith@NIKITH:~$ nano zombie.c  
nikith@NIKITH:~$ gcc zombie.c -o zombie  
nikith@NIKITH:~$ ./zombie  
Parent process: PID = 580  
Child process: PID = 581  
Parent sleeping...  
Child process: PID = 581  
Child PID = 581  
Child process terminating...  
Parent terminating...  
nikith@NIKITH:~$ |
```

TASK 2B

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <unistd.h>  
  
#include <sys/types.h>  
  
#include <sys/wait.h>
```

```
int main()  
{  
    pid_t pid;  
    int status;  
  
    pid = fork();  
  
    if (pid < 0)
```

```

{
    perror("fork failed");
    exit(EXIT_FAILURE);
}

if (pid == 0)
{
    // Child process
    printf("Child process: PID = %d\n", getpid());
    printf("Child PID = %d\n", getpid());

    printf("Child process terminating...\n");

    exit(0);
}
else
{
    // Parent process
    printf("Parent process: PID = %d\n", getpid());
    printf("Child process: PID = %d\n", pid);

    printf("Parent waiting for child using waitpid()...\n");

    // Parent waits for the specific child
    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        exit(EXIT_FAILURE);
    }
}

```

```

    if (WIFEXITED(status))
    {
        printf("Child exited normally with status %d\n",
               WEXITSTATUS(status));
    }

    printf("Parent terminating...\n");
}

return 0;
}

```

```

nikith@NIKITH:~$ nano zombie_fixed.c
nikith@NIKITH:~$ ^C
nikith@NIKITH:~$
nikith@NIKITH:~$ gcc zombie_fixed.c -o zombie_fixed
nikith@NIKITH:~$ ./zombie_fixed
Parent process: PID = 678
Child process: PID = 679
Parent waiting for child using waitpid()...
Child process: PID = 679
Child PID = 679
Child process terminating...
Child exited normally with status 0
Parent terminating...
nikith@NIKITH:~$ ps -el
 F S   UID     PID   PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
 4 S    0        1       0  0  80   0 -  6130 -        ?           00:00:00 system
 5 S    0        2       1  0  80   0 -   795 -        hvcc0       00:00:00 init-s
 0 S    0        6       2  0  80   0 -   795 -        hvcc0       00:00:00 init
 4 S    0       46       1  0  79  -1 - 10550 -        ?           00:00:00 system
 4 S   989       78       1  0  80   0 -   5635 -        ?           00:00:00 system
 4 S    0       84       1  0  80   0 -   8854 -        ?           00:00:00 system
 4 S    0      104       1  0  80   0 -    722 -        ?           00:00:00 chrony
 4 S    0      105       1  0  80   0 -   1118 -        ?           00:00:00 cron
 4 S   995      106       1  0  80   0 -   2198 -        ?           00:00:00 dbus-d
 4 S    0      137       1  0  80   0 -  11023 -        ?           00:00:00 networ
 4 S    0      162       1  0  80   0 -   4609 -        ?           00:00:00 system
 4 S    0      165       1  0  80   0 - 448075 -        ?           00:00:00 wsl-pr
 4 S   100      176       1  0  80   0 -  55137 -        ?           00:00:00 rsyslo
 4 S   988      215     104  0  80   0 -   5105 -        ?           00:00:00 chrony
 1 S   988      227     215  0  80   0 -   3023 -        ?           00:00:00 chrony
 4 S    0      282       1  0  80   0 -  30864 -        ?           00:00:00 unatte
 4 S    0      287       1  0  80   0 -   1373 -        tty1       00:00:00 agetty
 4 S    0      329       2  0  80   0 -   2162 -        ?           00:00:00 login
 4 S  1000      373       1  0  80   0 -   5583 -        ?           00:00:00 system
 5 S  1000      375     373  0  80   0 -   5727 -        ?           00:00:00 (sd-pa
 4 S  1000      401     329  0  80   0 -   1534 poll_s pts/1       00:00:00 bash
 5 S    0      514       2  0  80   0 -    796 -        ?           00:00:00 Sessio
 5 S    0      515     514  0  80   0 -    800 -        ?           00:00:00 Relay(
 4 S  1000      522     515  0  80   0 -   1567 do_wai pts/0       00:00:00 bash
 0 R  1000      689     522  0  80   0 -   1798 -        pts/0       00:00:00 ps
nikith@NIKITH:~$

```